

Aprendizaje No Supervisado

Introducción a Data Science — Clase 9

Ing. Juan Cruz Alric

Agenda del día

→ 01 — ¿Qué es el Aprendizaje No Supervisado?

Cuando los datos no tienen etiquetas y hay que descubrir patrones

→ 02 — Reglas de Asociación

Descubrir qué productos se compran juntos

→ 03 — K-Means Clustering

Agrupar datos en k grupos automáticamente

→ 04 — Clustering Jerárquico y DBSCAN

Más formas de agrupar: por jerarquía y por densidad

→ 05 — PCA: Reducción de Dimensionalidad

Comprimir información sin perder lo esencial

→ 06 — Panorama: ¿Cuándo uso qué?

Guía de decisión para elegir la técnica correcta

01

¿Qué es el Aprendizaje No Supervisado?

Encontrar estructura oculta en datos sin etiquetas

Objetivo de la sección

- Entender la **diferencia fundamental** entre supervisado y no supervisado
- Conocer los **3 tipos principales** de aprendizaje no supervisado
- Ver **ejemplos reales** de la industria
- Comprender el **flujo de trabajo** típico

La gran diferencia: con etiquetas vs sin etiquetas

 **Analogía:** Supervisado es como un examen con respuestas correctas — el modelo aprende comparando. No supervisado es como explorar una ciudad nueva **sin mapa** — descubrís patrones por tu cuenta.

Datos	Con etiquetas ($X \rightarrow y$)	Solo features (X)
Objetivo	Predecir una respuesta	Descubrir estructura
Ejemplo	¿Este email es spam?	¿Qué grupos de clientes tengo?
Evaluación	Accuracy, F1, RMSE	Silhouette, inercia, varianza
Feedback	Hay "respuesta correcta"	No hay respuesta correcta

Los 3 pilares del aprendizaje no supervisado

1. Clustering (Agrupamiento)

- Agrupar datos similares — K-Means, DBSCAN, Jerárquico
- "¿Qué grupos naturales existen en mis datos?"

2. Reglas de Asociación

- Descubrir relaciones entre ítems — Apriori, FP-Growth
- "¿Qué productos se compran juntos?"

3. Reducción de Dimensionalidad

- Comprimir muchas variables en pocas — PCA, t-SNE, UMAP
- "¿Puedo representar 50 columnas con solo 3?"

Ejemplos en la industria

Industria	Técnica	Aplicación
 Retail	Reglas de asociación	"Clientes que compran X también compran Y"
 Banca	Clustering	Segmentar clientes por perfil de riesgo
 Streaming	Clustering + PCA	Agrupar usuarios por gustos similares
 Salud	PCA + Clustering	Identificar subtipos de enfermedades
 Ciberseguridad	DBSCAN	Detectar tráfico de red anómalo
 Marketing	K-Means	Crear campañas por segmento de cliente

Flujo de trabajo típico

 **Analogía:** Es como un arquitecto que primero estudia el terreno (datos), luego organiza los materiales (features), y finalmente construye agrupaciones o reducciones.

Paso a paso:

- 1 **Recolectar datos** — sin etiquetas, solo features
- 2 **Explorar y limpiar** — EDA, nulos, outliers
- 3 **Escalar / normalizar** — ⚠ **fundamental** en no supervisado
- 4 **Elegir técnica** — clustering, asociación o reducción
- 5 **Ajustar parámetros** — k, eps, n_components...
- 6 **Evaluar resultados** — métricas internas + interpretación humana
- 7 **Iterar** — probar otras técnicas o parámetros

! El escalado NO es opcional

En aprendizaje no supervisado, **la mayoría de algoritmos usan distancias** (euclidiana, etc.)

→ Si una variable va de 0 a 1.000.000 y otra de 0 a 1, la primera **domina** todo el cálculo

→ Resultado: clusters basados solo en la variable de mayor escala

Siempre escalá antes de aplicar clustering o PCA:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

→ **StandardScaler** → media 0, desvío 1 (lo más común)

→ **MinMaxScaler** → rango [0, 1] (útil si necesitás rango fijo)

02

Reglas de Asociación

"Los que compraron pan, también compraron manteca"

Objetivo de la sección

- Entender qué son las **reglas de asociación** y para qué sirven
- Dominar las **3 métricas clave**: support, confidence, lift
- Conocer los **algoritmos** Apriori y FP-Growth
- Ver cómo se **aplican en retail y e-commerce**

¿Qué son las reglas de asociación?

 **Analogía:** Imaginá que sos el dueño de un supermercado y revisás miles de tickets. Notás que la gente que lleva **pan** casi siempre lleva **manteca**. Eso es una regla de asociación.

→ Buscan **patrones frecuentes** en conjuntos de transacciones

→ Formato: **A → B** ("si compran A, tienden a comprar B")

→ Aplicaciones:

-  Cross-selling en e-commerce
-  Diagnóstico médico (síntomas que aparecen juntos)
-  Análisis de navegación web (páginas visitadas juntas)

Las 3 métricas fundamentales

Support (Soporte)

→ ¿Qué tan frecuente es esta combinación?

→ $\text{support}(A \cap B) = \text{transacciones con A y B} / \text{total de transacciones}$

Confidence (Confianza)

→ Si alguien compra A, ¿qué probabilidad hay de que compre B?

→ $\text{confidence}(A \rightarrow B) = P(B|A) = \text{support}(A \cap B) / \text{support}(A)$

Lift (Elevación)

→ ¿La regla es mejor que el azar?

→ $\text{lift}(A \rightarrow B) = \text{confidence}(A \rightarrow B) / \text{support}(B)$

→ $\text{Lift} > 1$ = asociación positiva | $\text{Lift} = 1$ = independientes | $\text{Lift} < 1$ = negativa

Ejemplo concreto con números 1 2 3 4

Tenemos 100 transacciones en un supermercado.

Dato	Cantidad
Transacciones con Pan	30
Transacciones con Manteca	25
Transacciones con Pan Y Manteca	20

Calculamos la regla Pan → Manteca:

- $\text{Support}(\text{Pan} \wedge \text{Manteca}) = 20/100 = 0.20$ (20% de las transacciones)
- $\text{Support}(\text{Pan}) = 30/100 = 0.30$
- $\text{Confidence}(\text{Pan} \rightarrow \text{Manteca}) = 20/30 = 0.67$ (67% de los que compran pan, llevan manteca)
- $\text{Support}(\text{Manteca}) = 25/100 = 0.25$
- $\text{Lift}(\text{Pan} \rightarrow \text{Manteca}) = 0.67/0.25 = 2.67$ (🔥 fuerte asociación positiva!)

Interpretando el Lift

Lift	Significado	Acción
> 1	A y B se compran juntos más de lo esperado	✅ Regla útil — poner juntos
= 1	A y B son independientes	😐 No hay relación
< 1	A y B se compran juntos menos de lo esperado	🚫 Evitar cross-selling

En nuestro ejemplo, **lift = 2.67** significa que comprar pan hace 2.67 veces más probable que se compre manteca, **comparado con el azar**.

Apriori vs FP-Growth

Característica		
Estrategia	Genera candidatos nivel a nivel	Construye un árbol comprimido
Velocidad	Lento en datasets grandes	Mucho más rápido
Memoria	Múltiples pasadas por los datos	Una estructura en memoria
Uso	Didáctico, datasets pequeños	Producción, datasets grandes
Idea clave	"Si {A,B} no es frecuente, {A,B,C} tampoco"	Comprimir transacciones en un árbol

→ En la práctica, usá **FP-Growth** salvo que necesites entender el paso a paso

Código Python: Reglas de asociación (parte 1)

```
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder
import pandas as pd

# Datos: lista de transacciones
transacciones = [
    ['pan', 'manteca', 'leche'],
    ['pan', 'manteca'],
    ['leche', 'huevos'],
    ['pan', 'leche', 'huevos'],
    ['pan', 'manteca', 'huevos', 'leche'],
]

# Codificar en formato one-hot
te = TransactionEncoder()
te_array = te.fit(transacciones).transform(transacciones)
df = pd.DataFrame(te_array, columns=te.columns_)
```

Código Python: Reglas de asociación (parte 2)

```
# Encontrar itemsets frecuentes (support >= 0.4)
frecuentes = apriori(df, min_support=0.4, use_colnames=True)

# Generar reglas (confidence >= 0.6)
reglas = association_rules(frecuentes, metric="confidence",
                           min_threshold=0.6)
print(reglas[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

Concepto clave: Reglas de asociación

- **Support** te dice qué tan frecuente es la combinación
- **Confidence** te dice qué tan fuerte es la regla **en una dirección**
- **Lift** te dice si la regla es **mejor que el azar**

No te quedes solo con la confidence. Una regla con alta confidence pero bajo lift puede ser engañosa — tal vez B ya se compra mucho, con o sin A.

Siempre filtrá por las 3 métricas juntas para reglas de calidad.

03

K-Means Clustering

El algoritmo más popular para agrupar datos

Objetivo de la sección

- Entender **cómo funciona K-Means** paso a paso
- Conocer el problema de **inicialización** y cómo k-means++ lo resuelve
- Aprender a **elegir el número k** con Elbow y Silhouette
- Conocer las **limitaciones** del algoritmo

¿Qué hace K-Means?

🎯 **Analogía:** Imaginá que tenés 100 personas en un salón y querés formar **k grupos**. Ponés k banderas en el piso, cada persona camina hacia la bandera más cercana, y después movés cada bandera al centro de su grupo. Repetís hasta que nadie se mueve.

- **Entrada:** datos X y un número k (cantidad de clusters)
- **Salida:** cada dato asignado a uno de los k grupos
- **Criterio:** minimizar la distancia de cada punto a su centroide

K-Means paso a paso

Paso 1: Elegir k centroides iniciales

→ Se eligen k puntos al azar (o con k-means++)

Paso 2: Asignar cada punto al centroide más cercano

→ Cada dato se une al cluster del centroide más próximo

Paso 3: Recalcular centroides

→ El nuevo centroide es el **promedio** de todos los puntos del cluster

Paso 4: Repetir pasos 2 y 3

→ Hasta que los centroides **no se muevan más** (convergencia)

 Típicamente converge en 10–20 iteraciones. Siempre converge, pero puede caer en **mínimos locales**.

Visualizando K-Means: iteración a iteración

Iteración	¿Qué pasa?
Inicio	 Se eligen k centroides al azar
Iter 1	Cada punto se asigna al centroide más cercano → clusters iniciales
Iter 2	Centroides se mueven al centro de sus clusters → re-asignación
Iter 3	Menos puntos cambian de cluster → estabilizándose
...	Cada vez menos cambios
Final	 Ningún punto cambia → convergencia

 Es como remodelar una casa: cada pasada hace ajustes más pequeños hasta que todo está en su lugar.

El problema de la inicialización

¿Qué pasa si los centroides iniciales se eligen **mal**?

- K-Means puede converger a una **solución mala** (mínimo local)
- Dos ejecuciones distintas pueden dar **resultados diferentes**

Solución: **k-means++** 🌟

- En vez de elegir centroides totalmente al azar:
 1. Elegir el **primer centroide al azar**
 2. Elegir el siguiente **lo más lejos posible** de los ya elegidos
 3. Repetir hasta tener k centroides
- Resultado: centroides iniciales **bien separados** → mejor convergencia

```
# sklearn usa k-means++ por defecto
from sklearn.cluster import KMeans
modelo = KMeans(n_clusters=3, init='k-means++', n_init=10)
```

¿Cómo elegir k? El método del codo

El **Elbow Method** grafica la **inercia** (suma de distancias al centroide) para distintos valores de k.

- **Inercia alta** = pocos clusters, puntos lejos de su centroide
- **Inercia baja** = muchos clusters, puntos cerca de su centroide
- El "**codo**" es donde agregar más clusters ya no mejora mucho

```
import matplotlib.pyplot as plt

inercias = []
for k in range(1, 11):
    modelo = KMeans(n_clusters=k, random_state=42)
    modelo.fit(X_scaled)
    inercias.append(modelo.inertia_)

plt.plot(range(1, 11), inercias, 'bo-')
plt.xlabel('Número de clusters (k)')
plt.ylabel('Inercia')
plt.title('Método del Codo')
plt.show()
```

Coeficiente de Silhouette

Analogía: "¿Qué tan cómodo está cada punto en su cluster?" Si un punto está cerca de sus compañeros y lejos de los otros clusters, tiene alta silhouette.

→ Para cada punto se calcula: $s = (b - a) / \max(a, b)$

- **a** = distancia promedio a los puntos de **su propio cluster**
- **b** = distancia promedio al **cluster más cercano**

Valor	Significado
$s \approx 1$	Punto bien asignado, lejos de otros clusters
$s \approx 0$	Punto en la frontera entre dos clusters
$s < 0$	Punto probablemente en el cluster equivocado

→ **Silhouette promedio** te dice qué tan buenos son los clusters en general

Silhouette en Python

```
from sklearn.metrics import silhouette_score

mejores_scores = []
for k in range(2, 11):
    modelo = KMeans(n_clusters=k, random_state=42)
    labels = modelo.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    mejores_scores.append((k, score))
    print(f"k={k}, Silhouette={score:.3f}")

# Elegir k con mayor silhouette promedio
```

- Combiná **Elbow + Silhouette** para tomar la mejor decisión
- Si Elbow dice k=3 y Silhouette también es máximo en k=3 → 🎯

Limitaciones de K-Means

Limitación	Problema	Solución
Necesita k	Hay que decidir k de antemano	Elbow + Silhouette
Clusters esféricos	Asume clusters redondos y del mismo tamaño	Usar DBSCAN o GMM
Sensible a outliers	Un outlier puede desviar un centroide	Limpiar outliers antes
Sensible a escala	Variables con rangos grandes dominan	Escalar siempre
Mínimos locales	Puede converger a solución subóptima	k-means++, n_init=10

 K-Means es como una herramienta eléctrica: potente y rápido, pero no sirve para todo. Para formas irregulares, necesitás otra herramienta.

Ejercicio rápido: K-Means completo (parte 1)

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
import pandas as pd

# 1. Cargar y escalar
df = pd.read_csv('clientes.csv')
X = df[['edad', 'ingreso', 'gasto_anual']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Ejercicio rápido: K-Means completo (parte 2)

```
# 2. Encontrar mejor k
for k in range(2, 8):
    km = KMeans(n_clusters=k, random_state=42)
    labels = km.fit_predict(X_scaled)
    print(f"k={k} → Silhouette: {silhouette_score(X_scaled, labels):.3f}")

# 3. Entrenar con el mejor k
km_final = KMeans(n_clusters=3, random_state=42)
df['cluster'] = km_final.fit_predict(X_scaled)
```

04

Clustering Jerárquico y DBSCAN

Alternativas cuando K-Means no alcanza

Objetivo de la sección

- Entender el clustering **jerárquico aglomerativo** y el dendrograma
- Comprender **DBSCAN** y su enfoque basado en densidad
- Saber **cuándo usar cada uno** vs K-Means

Clustering Jerárquico Aglomerativo

🌳 **Analogía:** Imaginá que cada persona está sola en una mesa. Las dos personas más cercanas se juntan en una mesa. Después las dos mesas más cercanas se juntan. Y así hasta que todos están en una sola mesa gigante. El dendrograma es la foto de ese proceso.

¿Cómo funciona?

- 1 **Inicio:** Cada punto es su propio cluster (n clusters)
- 2 **Fusión:** Unir los 2 clusters más cercanos
- 3 **Repetir:** Seguir fusionando hasta que quede 1 solo cluster
- 4 **Cortar:** Elegir dónde "cortar" el dendrograma para obtener k clusters

Métodos de enlace (Linkage)

¿Cómo medimos la "distancia" entre dos clusters?

Método	¿Cómo mide la distancia?	Resultado
Single	Distancia entre los puntos más cercanos	Clusters alargados (cadena)
Complete	Distancia entre los puntos más lejanos	Clusters compactos
Average	Promedio de todas las distancias	Balance intermedio
Ward ★	Minimiza la varianza total al fusionar	Clusters esféricos, el más usado

→ En la práctica, **Ward** es el default más confiable

El Dendrograma

Es el "árbol genealógico" de los clusters — muestra cómo se fueron fusionando.

```
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

Z = linkage(X_scaled, method='ward')

plt.figure(figsize=(12, 6))
dendrogram(Z, truncate_mode='lastp', p=10)
plt.xlabel('Clusters')
plt.ylabel('Distancia')
plt.title('Dendrograma — Método Ward')
plt.axhline(y=5, color='r', linestyle='--', label='Corte sugerido')
plt.legend()
plt.show()
```

- ¿Dónde cortar? Donde haya un "salto grande" en la distancia vertical
- La línea roja horizontal indica el número de clusters resultante

Clustering Jerárquico en sklearn

```
from sklearn.cluster import AgglomerativeClustering

# Elegir n_clusters basado en el dendrograma
modelo = AgglomerativeClustering(n_clusters=3, linkage='ward')
df['cluster'] = modelo.fit_predict(X_scaled)

# Ver distribución de clusters
print(df['cluster'].value_counts())
```

- **Ventaja:** no necesitás elegir k de antemano (el dendrograma te guía)
- **Desventaja:** lento para datasets grandes ($O(n^3)$ en memoria)

DBSCAN: Clustering por densidad

🎉 **Analogía — "Amigos de amigos en una fiesta"**: Llegás a una fiesta. Si hay al menos 5 personas a menos de 2 metros de vos, sos parte de un "grupo". Tus amigos que también tienen 5 personas cerca se unen al mismo grupo. Los que están solos en un rincón son **ruido** (noise).

Los 2 parámetros clave:

Parámetro	Significado	Analogía
eps (ϵ)	Radio de búsqueda	"¿A qué distancia considero que alguien está cerca?"
min_samples	Mínimo de vecinos para ser "core"	"¿Cuántos amigos necesito cerca para ser un grupo?"

DBSCAN: Los 3 tipos de puntos

● Core point (punto núcleo)

- Tiene $\geq \text{min_samples}$ vecinos dentro de radio **eps**
- Es el "corazón" de un cluster

● Border point (punto frontera)

- Está dentro del radio eps de un core point, pero NO tiene suficientes vecinos propios
- Pertenece al cluster, pero en el "borde"

● Noise point (ruido)

- No es core ni border → **no pertenece a ningún cluster**
- DBSCAN lo etiqueta como **-1**

🎉 Volviendo a la fiesta: core = alma de la fiesta, border = alguien que está hablando con el alma de la fiesta, noise = el que está solo mirando el celular.

DBSCAN en Python

```
from sklearn.cluster import DBSCAN
import numpy as np

modelo = DBSCAN(eps=0.5, min_samples=5)
df['cluster'] = modelo.fit_predict(X_scaled)

# Ver clusters encontrados (-1 = ruido)
print(df['cluster'].value_counts())

# ¿Cuántos puntos de ruido?
n_ruido = (df['cluster'] == -1).sum()
print(f"Puntos de ruido: {n_ruido}")

# ¿Cuántos clusters reales?
n_clusters = len(set(df['cluster'])) - (1 if -1 in df['cluster'].values else 0)
print(f"Clusters encontrados: {n_clusters}")
```

→ DBSCAN **descubre k automáticamente** — no tenés que elegirlo

¿Cómo elegir eps y min_samples?

Para eps: **Gráfico k-distance**

```
from sklearn.neighbors import NearestNeighbors

nn = NearestNeighbors(n_neighbors=5)
nn.fit(X_scaled)
distances, _ = nn.kneighbors(X_scaled)

# Ordenar distancias al 5to vecino
dist_sorted = np.sort(distances[:, -1])
plt.plot(dist_sorted)
plt.xlabel('Puntos (ordenados)')
plt.ylabel('Distancia al 5º vecino')
plt.title('k-distance graph')
plt.show()
```

→ El "codo" en el gráfico te sugiere el valor de eps

Parámetros de DBSCAN

Para min_samples:

- Regla general: **min_samples $\geq$ dimensiones + 1**
- Más min_samples = más estricto (menos clusters, más ruido)

Comparación: K-Means vs Jerárquico vs DBSCAN

Característica			
Necesita k	✓ Sí	● Dendrograma ayuda	✗ No
Forma de clusters	Esféricos	Cualquiera	Cualquiera
Manejo de ruido	✗ No	✗ No	✓ Sí
Escalabilidad	✓ Muy rápido	✗ Lento ($O(n^3)$)	● Moderado
Clusters de distinto tamaño	✗	✓	✓
Determinístico	✗ (depende init)	✓	✓
Fácil de entender	✓	✓	●
Cuándo usarlo	Clusters claros y redondos	Exploración, pocas muestras	Formas irregulares, ruido

¿Cuál elijo?

- **Tenés idea de cuántos clusters** y datos "redondos" → **K-Means**
- **Querés explorar** la estructura jerárquica de los datos → **Jerárquico**
- **Tenés outliers** o clusters con formas raras → **DBSCAN**
- **Dataset enorme** (millones de filas) → **K-Means** (o Mini-Batch K-Means)
- **No sabés cuántos clusters** hay → **DBSCAN** o Jerárquico con dendrograma

No hay algoritmo "mejor" — hay algoritmo **adecuado para tu problema**.

05

PCA: Reducción de Dimensionalidad

Comprimir información sin perder lo esencial

Objetivo de la sección

- Entender **qué problema resuelve** la reducción de dimensionalidad
- Comprender **covarianza** de forma intuitiva
- Saber qué son **eigenvectores** y **eigenvalores** (sin miedo)
- Aprender a usar **PCA en Python** y elegir componentes
- Conocer las **limitaciones** de PCA

¿Por qué reducir dimensiones?

 **Analogía:** Tenés un plano arquitectónico con 100 vistas distintas del edificio. Pero con solo 3 vistas (frente, lateral, planta) ya podés entender el 95% de la estructura. PCA encuentra esas 3 "mejores vistas".

La maldición de la dimensionalidad 🐱

- Con muchas features, los puntos se "alejan" entre sí
- Los algoritmos de distancia (K-Means, KNN) **dejan de funcionar bien**
- Más columnas = más datos necesarios para que el modelo generalice
- Visualizar más de 3 dimensiones es imposible para humanos

PCA resuelve esto:

- Reducir 50 columnas a 3–5 **componentes principales**
- Conservando la **mayor cantidad de información** posible

Covarianza: la intuición

🎓 **Analogía:** Si cuando **altura** sube, **peso** también sube, tienen **covarianza positiva**. Si cuando **horas de estudio** suben, los **errores en examen** bajan, tienen **covarianza negativa**. Si no hay relación, la covarianza es **cercana a cero**.

Covarianza	Significado	Ejemplo
Positiva (+)	Suben y bajan juntas	Altura ↑ → Peso ↑
Negativa (-)	Una sube, la otra baja	Estudio ↑ → Errores ↓
Cerca de 0	No hay relación lineal	Altura ↔ Color favorito

→ PCA analiza la **matriz de covarianza** para encontrar las direcciones donde los datos varían más

Eigenvectores y Eigenvalores (sin miedo 😊)

💡 **Analogía de la linterna:** Imaginá que tenés una nube de puntos 3D y le apuntás con una linterna. La **sombra** que se proyecta en la pared es una versión 2D de tus datos. PCA busca el **mejor ángulo** para la linterna — el que produce la sombra más informativa (con más variación).

Traducción simple:

Concepto	¿Qué es?	Analogía
Eigenvector	La dirección de máxima variación	El ángulo de la linterna
Eigenvalor	La importancia de esa dirección	Qué tan "ancha" es la sombra
Componente principal	Eigenvector x datos	La sombra en la pared

→ El eigenvector con el eigenvalor más grande = la dirección más importante

→ El segundo eigenvector = la segunda dirección más importante (perpendicular)

PCA paso a paso

- 1 **Escalar los datos** — media 0, desvío 1 (StandardScaler)
- 2 **Calcular la matriz de covarianza** — relaciones entre todas las variables
- 3 **Obtener eigenvectores y eigenvalores** — direcciones e importancias
- 4 **Ordenar** por eigenvalor (de mayor a menor importancia)
- 5 **Elegir k componentes** — los que acumulen suficiente varianza
- 6 **Proyectar los datos** — transformar al nuevo espacio reducido

💡 En la analogía de la linterna: escalás los datos (1), analizás la forma de la nube (2–3), elegís los mejores ángulos (4–5), y proyectás la sombra (6).

PCA en Python (parte 1)

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

```
# 1. Escalar (OBLIGATORIO antes de PCA)
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
# 2. Aplicar PCA
```

```
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X_scaled)
```

PCA en Python (parte 2)

```
# 3. ¿Cuánta información conservamos?  
print(f"Varianza explicada por componente: {pca.explained_variance_ratio_}")  
print(f"Varianza total conservada: {sum(pca.explained_variance_ratio_):.2%}")
```

PCA en Python (parte 3)

```
# 4. Visualizar
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap='viridis')
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%})')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%})')
plt.title('Datos proyectados en 2 componentes principales')
plt.show()
```

¿Cuántos componentes elegir?

Buscamos conservar al menos el **80-95% de la varianza** total.

```
# PCA sin limitar componentes para ver la varianza acumulada
pca_full = PCA().fit(X_scaled)
varianza_acumulada = pca_full.explained_variance_ratio_.cumsum()

plt.plot(range(1, len(varianza_acumulada) + 1), varianza_acumulada, 'bo-')
plt.axhline(y=0.95, color='r', linestyle='--', label='95% varianza')
plt.xlabel('Número de componentes')
plt.ylabel('Varianza explicada acumulada')
plt.title('¿Cuántos componentes necesitamos?')
plt.legend()
plt.show()
```

- Si con **3 componentes** llegás al 95% de varianza, usá 3
- Si necesitás **20 de 50** para llegar al 95%, la reducción igual vale la pena

Limitaciones de PCA

Limitación	Explicación
Solo relaciones lineales	Si la relación entre variables es curva, PCA no la captura
Sensible a escala	⚠ Si no escalás, las variables grandes dominan
Pierde interpretabilidad	PC1 es una mezcla de variables originales — difícil de explicar
No maneja bien outliers	Un outlier extremo puede desviar los componentes
No es para todas las tareas	A veces las features originales son más útiles

Alternativas:

- **t-SNE**: para visualización 2D/3D (no lineal)
- **UMAP**: más rápido que t-SNE, preserva estructura global
- **Kernel PCA**: PCA no lineal (pero más lento)

PCA + K-Means: el combo poderoso (parte 1)

```
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# 1. Escalar
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 2. Reducir dimensionalidad
pca = PCA(n_components=0.95) # Conservar 95% varianza
X_pca = pca.fit_transform(X_scaled)
print(f"Dimensiones: {X.shape[1]} → {X_pca.shape[1]}")
```


PCA + K-Means: el combo poderoso (parte 2)

```
# 3. Clusterizar en el espacio reducido
km = KMeans(n_clusters=4, random_state=42)
labels = km.fit_predict(X_pca)

# 4. Visualizar (si 2D)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap='Set2')
plt.title('Clusters en espacio PCA')
plt.show()
```

→ PCA reduce ruido → K-Means funciona mejor 🎯

06

Panorama: ¿Cuándo uso qué?

Guía de decisión para elegir la técnica correcta

Objetivo de la sección

- Tener una **guía rápida** de cuándo usar cada técnica
- Comparar todas las técnicas en una **tabla resumen**
- Conocer los **parámetros clave** de cada una
- Saber qué **errores evitar**

Guía de decisión: ¿Qué técnica uso?

Si tu problema es...	Usá...	¿Por qué?
Agrupar clientes en segmentos claros	K-Means	Rápido, simple, clusters esféricos
Explorar jerarquía de grupos	Jerárquico	Dendrograma muestra estructura
Datos con outliers o formas raras	DBSCAN	Detecta ruido, formas arbitrarias
Descubrir productos que se compran juntos	Apriori / FP-Growth	Reglas de asociación
Reducir 50 features a pocas	PCA	Preserva varianza máxima
Visualizar datos de alta dimensión	PCA → 2D o t-SNE	Proyección a 2D
No sabés cuántos clusters hay	DBSCAN	Descubre k automáticamente
Dataset muy grande (millones)	Mini-Batch K-Means	Escalable

Tabla resumen: Parámetros clave (parte 1)

Técnica	Parámetros principales	Qué ajustar
K-Means	<code>n_clusters</code> , <code>init</code> , <code>n_init</code>	k (Elbow/Silhouette)
Jerárquico	<code>n_clusters</code> , <code>linkage</code>	Linkage + corte del dendrograma
DBSCAN	<code>eps</code> , <code>min_samples</code>	eps (k-distance), min_samples

Tabla resumen: Parámetros clave (parte 2)

Técnica	Parámetros principales	Qué ajustar
Apriori	<code>min_support</code>	Support mínimo
FP-Growth	<code>min_support</code>	Support mínimo
Reglas	<code>min_threshold</code> (confidence)	Confidence + Lift > 1
PCA	<code>n_components</code>	Varianza explicada $\geq 95\%$

Supuestos y limitaciones de cada técnica

Técnica	Asume...	Falla cuando...
K-Means	Clusters esféricos, misma densidad	Clusters alargados, outliers
Jerárquico	Estructura jerárquica en datos	Dataset muy grande (>10k)
DBSCAN	Clusters de densidad similar	Clusters con densidades muy distintas
Apriori	Transacciones discretas	Datos continuos (no aplica)
PCA	Relaciones lineales	Relaciones curvas/no lineales

Aplicaciones reales por industria

Industria	Clustering	Asociación	PCA
 Retail	Segmentar clientes	Market basket analysis	Reducir features de productos
 Banca	Perfiles de riesgo	Productos financieros relacionados	Reducir variables crediticias
 Salud	Subtipos de pacientes	Síntomas co-ocurrentes	Genómica (miles de genes)
 Gaming	Tipos de jugadores	Combinaciones de ítems	Embeddings de comportamiento
 Marketing	Audiencias similares	Canales que convierten juntos	Reducir encuestas largas

⚠ Errores comunes que debés evitar

1. No escalar los datos

→ K-Means y PCA **necesitan datos escalados**. Sin escalar, resultados basura.

2. Probar un solo valor de k

→ Siempre probá **múltiples valores** con Elbow + Silhouette.

3. Usar PCA sin escalar

→ Si no escalás, PCA solo captura la variable con mayor rango.

4. Ignorar el ruido en DBSCAN

→ Los puntos con label **-1** son ruido — **no los descartés sin analizarlos**.

5. Confiar solo en métricas

→ Siempre **visualizá los clusters** y verificá que tengan sentido de negocio.

6. Usar no supervisado cuando tenés etiquetas

Pipeline completo: de datos a insights

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# 1. Cargar datos
df = pd.read_csv('clientes.csv')
X = df.select_dtypes(include='number')

# 2. Escalar
X_scaled = StandardScaler().fit_transform(X)

# 3. Reducir dimensionalidad (si muchas columnas)
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_scaled)

# 4. Encontrar mejor k
for k in range(2, 8):
    labels = KMeans(n_clusters=k, random_state=42).fit_predict(X_reduced)
    print(f"k={k} → Silhouette: {silhouette_score(X_reduced, labels):.3f}")

# 5. Entrenar modelo final
km = KMeans(n_clusters=4, random_state=42)
df['segmento'] = km.fit_predict(X_reduced)

# 6. Analizar segmentos
print(df.groupby('segmento').mean())
```

Resumen ejecutivo

Clustering

- **K-Means**: rápido, simple, necesita k, clusters esféricos
- **Jerárquico**: dendrograma, exploratorio, lento con muchos datos
- **DBSCAN**: no necesita k, detecta ruido, formas arbitrarias

Reglas de Asociación

- **Apriori/FP-Growth**: descubrir ítems que co-ocurren
- Filtrar por **support + confidence + lift**

Reducción de Dimensionalidad

- **PCA**: comprimir features linealmente, elegir por varianza explicada
- **Siempre escalar antes**

Cheat Sheet: ¿Qué necesito?

Necesito...	Técnica	Código rápido
Agrupar en k grupos	K-Means	<code>KMeans(n_clusters=k)</code>
Agrupar sin elegir k	DBSCAN	<code>DBSCAN(eps=0.5, min_samples=5)</code>
Ver jerarquía de grupos	Jerárquico	<code>AgglomerativeClustering(n_clusters=k)</code>
Descubrir ítems relacionados	Apriori	<code>apriori(df, min_support=0.3)</code>
Reducir dimensiones	PCA	<code>PCA(n_components=0.95)</code>
Escalar datos	StandardScaler	<code>StandardScaler().fit_transform(X)</code>
Evaluar clusters	Silhouette	<code>silhouette_score(X, labels)</code>
Elegir k	Elbow	Graficar inercia vs k

Las 5 reglas de oro del no supervisado

1 Siempre escalá tus datos

StandardScaler es tu mejor amigo

2 Probá múltiples técnicas y parámetros

No te cases con el primer resultado

3 Visualizá los resultados

Los números mienten — los gráficos no (tanto)

4 Validá con conocimiento del dominio

Un cluster solo tiene valor si **significa algo** para el negocio

5 Iterá

Aprendizaje no supervisado es un proceso exploratorio, no una receta fija

¿Preguntas?

Introducción a Data Science — Clase 9

Ing. Juan Cruz Alric